

# Observabilité d'un jeu multijoueur

UE03 Données et Analytics  
Maxime Theophilos

Le service fourni reçoit les rapports de performance des clients et les fins de partie. Ce projet ajoute la collecte des logs, les métriques, les tableaux de bord, les alertes et le pipeline de publication.

## Pourquoi les joueurs ont-ils des saccades depuis jeudi soir ?

Les données indiquent une régression de l'overlay à partir de **beta-20260924-3**, observée le jeudi 24 septembre. Le premier rapport cohérent de cette build arrive à 19:06:36, puis le premier rapport overlay à 20:16:26. Ces heures sont celles des observations, pas celles d'un déploiement connu.

Les configurations surtout concernées ont une largeur d'au moins 2 560 pixels et le bloom actif. La part des rapports overlay y passe de moins de 1 % à 61,59 %, puis 63,04 % sur la build suivante. Dans ce groupe, 40 identifiants client distincts ont envoyé 249 rapports overlay sur la période. Le coût se concentre côté CPU. L'export ne donne pas le nombre total de joueurs exposés.

Requêtes dans docs/queries.sql : build\_first\_valid, overlay\_timeline, build\_cohorts, overlay\_context.

## Est-ce que quelqu'un abuse du jeu ?

Les 83 victoires très courtes sur vault, regroupées en deux vagues nocturnes, font penser à du farming. Il faut vérifier les actions et les résultats côté serveur avant d'attribuer un abus. Un autre client, 5e1f0c7a, envoie 70 rapports incohérents : une falsification ou un défaut d'instrumentation sont possibles. Les retards de bots sur foundry constituent un problème distinct, compatible avec un dysfonctionnement serveur.

Requêtes dans docs/queries.sql : short\_wins, short\_win\_clients, invalid\_clients, bots.

## Organisation du projet

| Données                     | Traitement                                                                                                                           |
|-----------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| API et simulation en direct | Prometheus collecte les métriques. Alloy transmet les logs à Loki. Grafana permet de consulter les deux.                             |
| Export historique fourni    | L'import normalise les événements et les envoie à Loki avec leurs dates d'origine. Les mêmes événements alimentent l'analyse SQLite. |
| Construction et publication | GitHub Actions lance les contrôles, construit l'image, recherche les vulnérabilités critiques et publie dans GHCR.                   |

# 1. Qualité des données

L'export contient des blocs JSON précédés d'un en-tête, et non un événement JSON par ligne.

L'import conserve les lignes sources, déduplique les événements et les classe par date. Les heures de ce rapport sont en Europe/Paris.

Chaque nom de requête ci-dessous correspond à une requête exécutable dans docs/queries.sql.

| Contrôle         | Résultat et traitement                                                                                                                        | Requête                    |
|------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|----------------------------|
| Volume et format | 2 944 blocs lus, aucun JSON rejeté. Après déduplication : 2 110 rapports et 779 fins de partie, soit 2 889 événements.                        | audit<br>volumes           |
| Doublons         | 55 doublons exacts : 38 rapports et 17 fins. La clé combine type et identifiant. Aucun conflit de contenu dans cet export.                    | audit                      |
| Horodatage       | Du 20/09 à 00:31:37 au 27/09 à 00:04:13. Les 6 événements après minuit sont conservés. Conversion des en-têtes AM/PM de Paris vers UTC.       | time_range<br>daily_counts |
| Ordre            | Export décroissant avec 2 inversions locales de 1 s et 3 s. Tri croissant pour Loki, sans perdre la ligne source.                             | export_order               |
| Schéma           | 2 085 rapports v1 et 25 v2. Le RTT passe de rttMs à network.rttMs. Aucun RTT perdu après normalisation.                                       | schema_rtt                 |
| GPU              | 882 mesures absentes sur 2 110 rapports, toutes avec gpuTimerSupported=false. Une absence ne devient pas une durée nulle.                     | gpu_missing                |
| Cohérence        | 70 rapports incohérents (3,32 %) avec durées ou sommes impossibles. Ils restent dans les logs et sont exclus des statistiques de performance. | quality_issues<br>causes   |
| Somme des stages | En v2, physics ajoute 0,4 à 1,4 ms sans entrer dans totalMs. Il reste validé mais est exclu de cette somme. Tolérance d'arrondi : 3 ms.       | v2_stage_difference        |
| Autres contrôles | Aucun rapport avant la création de partie, après une fin connue, ou avec un identifiant d'en-tête différent du contenu.                       | invariants                 |

**Trou de collecte** : 2 h 15 min 11 s sans événement le 22 septembre, de 13:59:23 à 16:14:34. Cela ne prouve pas une panne : l'export ne contient pas de signal périodique de disponibilité. Requête : largest\_gaps.

**Fins manquantes** : 4 rapports cohérents concernent 2 parties sans fin associée, auxquels s'ajoutent les 70 rapports incohérents liés à 70 autres identifiants serveur. Requête : missing\_completions.

L'import accepte aussi BOM et CRLF, couverts par les tests mais absents de cet export. En cas d'identifiant réutilisé avec un contenu différent, il conserve le premier événement, note le conflit et sort en erreur.

## 2. Classification des rapports

Chaque rapport reçoit une cause principale. L'ordre de priorité est : invalid, hidden, overlay, shader, world, network, render, unknown. Les signatures secondaires restent disponibles. Les retards de bots et les victoires courtes sont détectés séparément.

| Cause   | Nombre | Condition et interprétation                                                                                                                      |
|---------|--------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| invalid | 70     | Type, signe ou somme des temps incohérents. Ces données ne sont pas utilisées pour mesurer la performance.                                       |
| hidden  | 158    | Onglet masqué et délai navigateur $\geq 800$ ms. Ce ralentissement peut venir de la limitation des onglets en arrière-plan.                      |
| overlay | 262    | renderOverlay $\geq 200$ ms. La comparaison par build et configuration permet ensuite de rechercher une régression.                              |
| shader  | 459    | Première capture et au moins 40 nouveaux programmes graphiques. Cela suggère une compilation de shaders au chargement.                           |
| world   | 20     | worldDynamics $\geq 60$ ms. Les 20 cas appartiennent ici au groupe de parties avec des bots en retard.                                           |
| network | 92     | reason=network et (RTT $\geq 180$ ms ou intervalle maximal entre ticks $\geq 150$ ms). Il faut comparer les deux valeurs pour préciser la cause. |
| render  | 1 049  | Stage render positif sans signature précédente. Le classement ne suffit pas à expliquer le ralentissement.                                       |
| unknown | 0      | Aucune condition reconnue. Cette catégorie conserve les cas qui ne correspondent pas aux signatures prévues.                                     |

Requêtes dans docs/queries.sql : causes, quality\_issues, bots, bot\_comparaison.

### Choix des seuils

Les seuils se trouvent dans config/detection.json. Ils séparent les groupes observés dans l'export : plusieurs centaines de millisecondes pour l'overlay, forte création de shaders, ou retard de décision des bots de plusieurs secondes. Les détections de bots utilisent au moins 2 bots en retard et un âge d'au moins 1 000 ms.

Ces seuils restent à vérifier sur du trafic complet. Le jeu remonte ici des pics de performance, pas toutes ses frames. La répartition des causes décrit donc les rapports reçus, pas la fréquence des problèmes chez tous les joueurs.

### Contrôles du traitement

Les tests couvrent les deux versions de schéma, les dates de Paris, les doublons, les conflits et les blocs tronqués. Ils vérifient aussi les durées négatives, les chaînes à la place de nombres et les sommes impossibles. Le même code de classification traite l'historique et les événements en direct.

Fichiers : src/export-parser.js, src/classify.js et test/analysis.test.js.

### 3. Métriques et collecte

#### Métriques ou logs ?

Les métriques décrivent le service en direct : débit, erreurs, temps de réponse et compteurs par cause ou build. Prometheus les collecte toutes les 10 secondes et évalue les règles toutes les 15 secondes.

Les logs gardent le détail d'un événement : client, partie, contenu du rapport et ligne source pour l'import. Ils permettent de retrouver une trace précise dans Loki. L'historique y garde ses dates d'origine. Il n'alimente pas les compteurs actuels de Prometheus, car le rejouer comme du trafic neuf fausserait le débit et les alertes.

| Métrique                                                      | Type et usage                                                                          | Labels                |
|---------------------------------------------------------------|----------------------------------------------------------------------------------------|-----------------------|
| game_http_requests_total                                      | Counter. Réponses terminées, débit avec rate() et erreurs 5xx.                         | method, route, status |
| game_http_duration_seconds                                    | Histogram. Temps de réponse en secondes, classes de 5 ms à 2 s, calcul du p95.         | method, route         |
| game_active                                                   | Gauge. Nombre de parties en cours, qui augmente et diminue.                            | Aucun                 |
| game_reports_total                                            | Counter. Rapports reçus, y compris invalid et hidden.                                  | source, build, cause  |
| game_frame_duration_seconds                                   | Histogram. Frames signalées hors invalid, hidden et unknown ; classes de 16 ms à 5 s.  | source, build, cause  |
| game_completed_total<br>game_short_wins_total                 | Counters. Fins de partie et victoires courtes. La durée simulée tient compte de SPEED. | map / aucun           |
| game_server_gap_reports_total<br>game_stale_bot_reports_total | Counters. Rapports cohérents avec intervalle entre ticks ou retard de bots excessif.   | source / source, map  |

Les labels restent limités : 2 sources, 5 builds connues plus other et 8 causes, soit 96 combinaisons pour le compteur de rapports. Les routes inconnues deviennent unmatched. Aucun identifiant client ou serveur n'est un label Prometheus.

#### Règles de calcul et collecte des logs

Les recording rules calculent le débit (game:api\_request\_rate:5m), la part de 5xx (game:api\_error\_ratio:5m) et le p95 (game:api\_latency\_p95:5m). Le débit exclut /healthz et /metrics ne compte pas sa propre lecture. Le p95 est une estimation calculée avec histogram\_quantile, selon la largeur des classes.

Dans Loki, seuls service\_name, dataset, event et source sont indexés. Les autres champs sont extraits lors de la requête. Alloy lit le JSONL et conserve sa position. Un import identique préserve le fichier et son inode. Un autre export doit utiliser un jeu de données distinct.

## 4. Régression de l'overlay

Le groupe étudié réunit les rapports cohérents avec une largeur d'au moins 2 560 pixels et le bloom actif. Le tableau compare ce groupe aux autres configurations. Les effectifs sont des nombres de rapports.

| Build           | ≥ 2 560 px et bloom  | Autres configurations |
|-----------------|----------------------|-----------------------|
| beta-20260919-2 | 1 / 180<br>0,56 %    | 4 / 461<br>0,87 %     |
| beta-20260922-1 | 0 / 137<br>0,00 %    | 4 / 488<br>0,82 %     |
| beta-20260924-3 | 186 / 302<br>61,59 % | 4 / 276<br>1,45 %     |
| beta-20260926-5 | 58 / 92<br>63,04 %   | 1 / 79<br>1,27 %      |
| beta-20260926-6 | 4 / 6<br>66,67 %     | 0 / 19<br>0,00 %      |

Requêtes dans docs/queries.sql : build\_cohorts.

La hausse apparaît sur beta-20260924-3 et persiste sur beta-20260926-5. Hors de ce groupe, la part d'overlay reste entre 0 et 1,45 %. Les 6 rapports du groupe sur beta-20260926-6 sont trop peu nombreux pour conclure à un rétablissement ou à une aggravation.

### Population et coût observés

Les 249 rapports overlay du groupe proviennent de **40 identifiants client distincts**, répartis entre Brave, Chrome, Edge, Firefox et Safari. Le coût médian de renderOverlay est de **434 ms**, soit une part médiane de **90,4 % du travail CPU**. Le RTT médian est de 34,8 ms et l'intervalle maximal moyen entre ticks est proche de 51 ms dans chaque famille de navigateurs.

Ces résultats orientent vers le rendu CPU de l'overlay plutôt que vers une panne réseau générale. Seuls 118 rapports disposent d'une mesure GPU, âgée de 107,6 à 884,5 ms. Elle ne correspond donc pas forcément à la frame signalée.

Requêtes dans docs/queries.sql : overlay\_context, overlay\_population.

### Exemple de trace

Rapport P-41a28e20-18, ligne 106142, le 24 septembre à 20:16:26 : largeur 3 440 px, bloom actif, frame de 523 ms dont 471,1 ms pour l'overlay. RTT de 58,4 ms et intervalle maximal entre ticks de 50,96 ms.

Requêtes dans docs/queries.sql : overlay\_evidence.

La capture du panneau de comparaison figure dans la section 9. Le panneau des navigateurs utilise tous les rapports overlay (262), tandis que cette analyse porte sur les 249 de la configuration étudiée.

## 5. Chronologie et actions correctives

Les premières observations situent la régression le jeudi 24 septembre au soir. Elles ne donnent pas l'heure exacte de mise en production. Les heures ci-dessous sont en Europe/Paris.

| Date et heure      | Observation                                                                                                          |
|--------------------|----------------------------------------------------------------------------------------------------------------------|
| 20 au 24 septembre | 9 rapports overlay sur les deux anciennes builds. Le problème existe ponctuellement avant la hausse.                 |
| 24/09 09:35:45     | Premier rapport annonçant beta-20260924-3, mais incohérent. Il est écarté pour dater la diffusion de cette build.    |
| 24/09 19:06:36     | Premier rapport cohérent de beta-20260924-3.                                                                         |
| 24/09 20:16:26     | Premier rapport overlay de beta-20260924-3, décrit dans la section précédente.                                       |
| 26/09 12:32:23     | Premier rapport cohérent de beta-20260926-5. La part d'overlay reste élevée dans le groupe étudié : 63,04 %.         |
| 26/09 20:50:28     | Premier rapport cohérent de beta-20260926-6, au format v2. L'échantillon ne permet pas de conclure à une résolution. |

Requêtes dans docs/queries.sql : build\_first\_valid, overlay\_timeline, invalid\_clients.

### Cause probable et impact

La comparaison des builds, des configurations et des temps de calcul indique un chemin CPU de l'overlay devenu coûteux en grande résolution avec bloom. Une frame de plusieurs centaines de millisecondes provoque une saccade visible. Les 40 identifiants client observés ne représentent pas tous les joueurs exposés : l'export ne couvre que les rapports reçus.

Le générateur fourni reproduit cette condition. Il permet de tester la collecte et les détections, mais ne suffit pas à prouver quel changement de code client aurait été déployé. Il manque aussi une mesure après correction.

### Actions à mener

| Priorité   | Action                                                                                          | Vérification                                                                                     |
|------------|-------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| Immédiate  | Tester la désactivation du bloom ou de l'overlay dans les configurations touchées.              | Comparer coût CPU et fréquence des pics avec et sans cette option.                               |
| Correction | Profiler l'overlay à 2 560, 3 440 et 3 840 px. Comparer les builds et limiter les recalculs.    | Rejouer les mêmes scènes sur le même matériel avant et après la correction.                      |
| Prévention | Ajouter des tests de durée de frame et déployer d'abord sur un groupe limité de configurations. | Vérifier un volume suffisant de rapports, les autres configurations et la stabilité de l'alerte. |

## 6. Abus possibles et autres incidents

### Victoires courtes sur vault

**83 parties** se terminent sur le score de 3-0 en 54,349 à 74,430 secondes. Il y a deux vagues : 45 parties le 23 septembre de 01:14:22 à 03:46:35, puis 38 le 25 de 00:44:29 à 02:32:54. Seules 17 sont en quarantaine. Le seuil de 100 secondes isole ces parties des suivantes, dont la durée commence à environ 171 secondes.

La répétition suggère du farming. Les jointures relient 14 parties à trois clients : 7247704e (7), 5db9ad62 (5) et 7440b319 (2). Les 69 autres n'ont pas de rapport client associé. Il faut vérifier les actions, les adversaires et les conditions de victoire côté serveur avant toute attribution ou sanction.

Requêtes dans docs/queries.sql : short\_wins, short\_win\_clients.

### Retards de bots sur foundry

**91 rapports** issus de 32 clients concernent 33 parties foundry : 13 le 21 septembre et 20 le 24. Deux ou trois bots ont des décisions âgées de 2 058 à 8 992 ms, contre 18 à 45 ms ailleurs. Toutes ces parties finissent 3-0 en 170,815 à 282,434 secondes. Les 20 rapports classés world appartiennent à ce groupe.

Ces retards suggèrent un dysfonctionnement du traitement des bots. Ils ne démontrent pas un abus du joueur. Il faut comparer la charge serveur, le temps worldDynamics et la file de décisions, puis vérifier leur progression dans un test de charge.

Requêtes dans docs/queries.sql : bots, bot\_comparison.

### Rapports incohérents du client 5e1f0c7a

Ce client envoie **70 rapports** associés à 70 identifiants serveur, sans fin de partie correspondante. Les données annoncent jusqu'à 1 200 FPS, un temps render de -101,5 ms et des sommes impossibles. Elles annoncent toutes beta-20260924-3, dès le 24 septembre à 09:35:45, environ 9 h 30 avant le premier rapport cohérent de cette build.

Une falsification est possible, mais un défaut d'instrumentation reste à vérifier. Il faut isoler ces rapports, contrôler leur schéma et leur provenance, et conserver les données brutes. Le contenu seul ne prouve pas l'intention de leur auteur.

Requêtes dans docs/queries.sql : invalid\_clients, missing\_completions, build\_first\_valid.

### Intervalles entre ticks trop longs

16 rapports cohérents ont un tickGapMaxMs d'au moins 150 ms, le 21 septembre dans les tranches de 02 h et 03 h, puis le 23 autour de 03 h. Le maximum atteint 746,6 ms. Seuls 7 des 92 rapports classés network appartiennent à cet ensemble. tickGapMaxMs mesure un intervalle entre ticks, pas leur durée d'exécution.

Requêtes dans docs/queries.sql : server\_degradation, causes.

## 7. Alertes

Les règles sont dans `config/prometheus/rules.yml`. Elles portent sur le trafic en direct. Les seuils déclenchent une vérification, pas une sanction automatique.

| Alerte               | Condition et durée                                                                                          | Limite                                                                                                  |
|----------------------|-------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|
| TelemetryUnavailable | up=0 pendant 1 min.                                                                                         | Un redémarrage peut suffire. Une cible supprimée de la configuration échappe à cette règle.             |
| ApiErrors            | Plus de 5 % de 5xx avec un débit > 1 req/s, pendant 2 min.                                                  | Une panne à très faible trafic peut passer inaperçue.                                                   |
| ApiLatency           | p95 > 250 ms avec un débit > 1 req/s, pendant 2 min.                                                        | Une rafale locale ou un CPU partagé peut ralentir les réponses.                                         |
| OverlayRegression    | Plus de 20 % d'overlay hors invalid/hidden sur 5 min, au moins 20 rapports, pendant 2 min.                  | L'échantillonnage des pics biaise la proportion. Un petit groupe peut rester invisible.                 |
| InvalidTelemetry     | Au moins 3 rapports invalides sur 5 min, pendant 1 min.                                                     | Un bug de schéma peut ressembler à une falsification.                                                   |
| ShortWins            | Au moins 5 victoires vault 3-0 de moins de 100 s sur 5 min, pendant 1 min.                                  | Des parties légitimes ou des tests peuvent déclencher la règle. Un farming plus lent peut lui échapper. |
| ServerTickDegraded   | Au moins 5 rapports cohérents avec <code>tickGapMaxMs</code> ≥ 150 ms sur 5 min, pendant 1 min.             | Plusieurs rapports peuvent décrire une même partie. Un incident sans rapport reste invisible.           |
| StaleBotDecisions    | Au moins 5 rapports cohérents avec au moins 2 bots en retard et un âge ≥ 1 000 ms sur 5 min, pendant 1 min. | Un serveur peut être surreprésenté. Sans rapport client, le défaut n'est pas détecté.                   |

### Justification et validation

Les fenêtres de 5 minutes et les délais de 1 à 2 minutes limitent les déclenchements sur un événement isolé. Le minimum de 20 rapports évite qu'un seul overlay donne une proportion de 100 %. Le seuil de 20 % reste un point de départ, à ajuster sur du trafic complet.

Les seuils de 100 s, 1 000 ms et 150 ms séparent les groupes observés : victoires de 54 à 74 s, décisions de bots vieilles de plus de 2 s, et intervalles entre ticks de plusieurs centaines de millisecondes. Les tests promtool couvrent 21 scénarios, 32 assertions d'alertes et 8 assertions PromQL.

### Alertes pendant la démonstration

Le générateur de charge produit environ 25 % d'overlay parmi les rapports hors onglets masqués. OverlayRegression peut donc être active avec `source=ingest`. Le simulateur de `src/fleet.js` injecte aussi des incidents `tickDegraded` : `ServerTickDegraded` peut alors apparaître avec `source=simulation`. Leur état dépend du trafic récent. Les alertes sont visibles dans Grafana et Prometheus ; aucune notification externe n'est configurée.

## 8. Docker et intégration continue

### Construction et démarrage

Le Dockerfile installe les dépendances dans une première étape. L'image finale ne garde que les dépendances de production, le code et la configuration nécessaires. La base Node 22 Alpine est épinglée par digest. Le processus tourne avec l'utilisateur node.

Compose utilise une racine en lecture seule, retire les capacités et expose les ports sur 127.0.0.1. La commande `docker compose up --build` lance sept services. Le service history termine l'import puis s'arrête avec le code 0. Les six autres continuent de fonctionner.

### Contrôles avant publication

```
lint -> tests -> import et analyse -> promtool
-> construction -> scan CRITICAL -> publication GHCR
```

Le job image dépend du succès de validate. Trivy analyse une archive de l'image et sort avec le code 1 si une vulnérabilité CRITICAL est trouvée. Seules les vulnérabilités de ce niveau sont recherchées par ce contrôle. La publication se fait sur main, hors pull request et hors scénario de démonstration. Le tag contient le SHA complet du commit.

| Scénario               | Résultat                                                                  | Exécution GitHub                   |
|------------------------|---------------------------------------------------------------------------|------------------------------------|
| Normal                 | 18 tests réussis, scan sans CRITICAL, image publiée.                      | <a href="#"><u>37634572207</u></a> |
| Test en échec          | 17 tests réussis et 1 échec volontaire. Le job image est ignoré.          | <a href="#"><u>37634867100</u></a> |
| Vulnérabilité critique | CVE-2019-10744 dans lodash 4.17.11. Scan en échec et publication ignorée. | <a href="#"><u>37634872199</u></a> |

Ces trois exécutions ont été vérifiées le 7 octobre sur le commit 03a85e5. La dépendance vulnérable est ajoutée uniquement à une image de démonstration. Selon le scanner, la correction de cette CVE apparaît dans lodash 4.17.12.

Image publiée :

```
ghcr.io/thekeyyn/maxime_rattrapage:sha-03a85e53c34ce0c2411eb2508f527f44d2f777b2
```

Les liens sont dans `evidence/ci/runs.md`. Les journaux, les rapports Trivy et le digest du manifeste sont dans `evidence/ci/github/`. Le scan dépend de la base de vulnérabilités et ne garantit pas l'absence de tout défaut de sécurité.

### Autres preuves

`evidence/tests.log` contient les tests Node, `evidence/promtool.log` les contrôles de règles, et `evidence/stack-verification.json` la vérification des 28 requêtes des dashboards. Les captures complètes sont dans `evidence/screenshots/`.

## 9. Captures Grafana

Les trois dashboards sont provisionnés depuis `config/grafana/`. Chaque panneau pose une question. Les deux extraits suivants montrent les principaux résultats de l'enquête historique.

### Part des rapports overlay par build



Source : `evidence/screenshots/performance-cohortes.jpg`. Le panneau porte sur les rapports cohérents avec largeur  $\geq 2\,560$  px et bloom actif. Les valeurs arrondies correspondent au tableau de la section 4.

Requêtes dans `docs/queries.sql` : `build_cohorts`.

### Client aux rapports incohérents

Quels clients envoient des données incohérentes ? ⓘ

| Client   | Rapports incohérents |
|----------|----------------------|
| 5e1f0c7a | 70                   |

Source : `evidence/screenshots/integrite-details.jpg`. Les 70 rapports incohérents appartiennent au même identifiant client.

Requêtes dans `docs/queries.sql` : `invalid_clients`.

Les captures complètes conservent la période affichée et les autres panneaux. Les totaux historiques attendus sont de 2 110 rapports et 779 fins de partie. Le tableau de santé du service suit, lui, les métriques de la simulation en cours : ses valeurs changent pendant la démonstration.

## 10. Reproduction et sources

### Démarrer le projet

```
docker compose up --build -d  
docker compose ps --all
```

Grafana est disponible sur localhost:3000 et Prometheus sur localhost:9090. Le README donne le compte local, les liens des dashboards et la période à sélectionner pour l'historique.

### Rejouer les contrôles locaux

```
npm ci --ignore-scripts  
npm run verify  
npm run import:history  
npm run analyze  
python3 scripts/verify-stack.py
```

Node 22.13 ou supérieur, Python 3 et Docker sont nécessaires. L'analyse exécute les 25 requêtes de docs/queries.sql et écrit artifacts/analysis.json et artifacts/analysis.sqlite. La vérification de stack interroge la stack en cours et met à jour evidence/stack-verification.json.

### Documentation utilisée

[Prometheus : histogrammes](#)

[Prometheus : tests des règles](#)

[Loki : cardinalité des labels](#)

[Trivy : code de sortie](#)